Programação Orientada a Objetos

Leonardo Vitazik Neto

Católica de Santa Catarina
Centro Universitário

Agenda

- Controle de fluxo:
  - O IF E O ELSE;
  - WHILE;
  - FOR;
  - BREAK;
  - CONTINUE;
  - ESCOPO DAS VARIÁVEIS;
- Exercício.

O IF E O ELSE

A sintaxe do IF no Java é a seguinte:

```
if (condicaoBooleana) {
    codigo;
}
```

Uma **condição booleana** é qualquer expressão que retorne true ou false . Para isso, você pode usar os operadores < , > , <= , >= e outros. Um exemplo:

```
int idade = 15;
if (idade < 18) {
    System.out.println("Não pode entrar");
}
```

Além disso, você pode usar a cláusula else para indicar o comportamento que deve ser executado no caso da expressão booleana ser falsa:

```
int idade = 15;
if (idade < 18) {
    System.out.println("Não pode entrar");
} else {
    System.out.println("Pode entrar");
}
```

O IF E O ELSE

Você pode concatenar expressões booleanas através dos operadores lógicos **"E"** e **"OU"**.
O **"E"** é representado pelo `&&` e o **"OU"** é representado pelo `||`.

Um exemplo seria verificar se ele tem menos de 18 anos **e** se ele não é amigo do dono:

```
int idade = 15;
boolean amigoDoDono = true;
if (idade < 18 && amigoDoDono == false) {
    System.out.println("Não pode entrar");
}
else {
    System.out.println("Pode entrar");
}
```

O IF E O ELSE

Esse código poderia ficar ainda mais legível, utilizando-se o operador de negação, o ! . Esse operador transforma o resultado de uma expressão booleana de false para true e vice versa.

```
int idade = 15;
boolean amigoDoDono = true;
if (idade < 18 && !amigoDoDono) {
    System.out.println("Não pode entrar");
}
else {
    System.out.println("Pode entrar");
}
```

Repare na linha 3 que o trecho amigoDoDono == false virou !amigoDoDono . Eles têm o mesmo valor.

Para comparar se uma variável tem o mesmo valor que outra variável ou valor, utilizamos o operador == . Repare que utilizar o operador = dentro de um if vai retornar um erro de compilação, já que o operador = é o de atribuição.

```
int mes = 1;
if (mes == 1) {
    System.out.println("Você deveria estar de férias");
}
```

O WHILE

O while é um comando usado para fazer um laço (loop), isto é, repetir um trecho de código algumas vezes. A ideia é que esse trecho de código seja repetido enquanto uma determinada condição permanecer verdadeira.

```
int idade = 15;
while (idade < 18) {
    System.out.println(idade);
    idade = idade + 1;
}
```

O trecho dentro do bloco do while será executado até o momento em que a condição idade < 18 passe a ser falsa. E isso ocorrerá exatamente no momento em que idade == 18 , o que não o fará imprimir 18 .

```
int i = 0;
while (i < 10) {
    System.out.println(i);
    i = i + 1;
}
```

Já o while acima imprime de 0 a 9.

O FOR

Outro comando de loop extremamente utilizado é o for . A ideia é a mesma do while : fazer um trecho de código ser repetido enquanto uma condição continuar verdadeira. Mas além disso, o for isola também um espaço para inicialização de variáveis e o modificador dessas variáveis. Isso faz com que fiquem mais legíveis, as variáveis que são relacionadas ao loop:

```
for (inicializacao; condicao; incremento) {
    codigo;
}
```

Um exemplo é o a seguir:

```
for (int i = 0; i < 10; i = i + 1) {
    System.out.println("olá!");
}
```

Repare que esse for poderia ser trocado por:

```
int i = 0;
while (i < 10) {
    System.out.println("olá!");
    i = i + 1;
}
```

Porém, o código do for indica claramente que a variável i serve, em especial, para controlar a quantidade de laços executados. Quando usar o for ? Quando usar o while ? Depende do gosto e da ocasião.

O FOR

Pós incremento ++

`i = i + 1` pode realmente ser substituído por `i++` quando isolado, porém, em alguns casos, temos essa instrução envolvida em, por exemplo, uma atribuição:

```
int i = 5;
int x = i++;
```

Qual é o valor de `x` ? O de `i` , após essa linha, é 6.

O operador `++` , quando vem após a variável, retorna o valor antigo, e incrementa (pós incremento), fazendo `x` valer 5.

Se você tivesse usado o `++` antes da variável (pré incremento), o resultado seria 6:

```
int i = 5;
int x = ++i; // aqui x valera 6
```

CONTROLANDO LOOPS (BREAK)

Apesar de termos condições booleanas nos nossos laços, em algum momento, podemos decidir parar o loop por algum motivo especial sem que o resto do laço seja executado.

```
for (int i = x; i < y; i++) {
    if (i % 19 == 0) {
        System.out.println("Achei um número divisível por 19 entre x e y");
        break;
    }
}
```

O código acima vai percorrer os números de x a y e parar quando encontrar um número divisível por 19, uma vez que foi utilizada a palavra chave break.

CONTROLANDO LOOPS (CONTINUE)

Da mesma maneira, é possível obrigar o loop a executar o próximo laço. Para isso usamos a palavra chave continue .

```
for (int i = 0; i < 100; i++) {
    if (i > 50 && i < 60) {
        continue;
    }
    System.out.println(i);
}
```

O código acima não vai imprimir alguns números. (Quais exatamente?)

ESCOPO DAS VARIÁVEIS

No Java, podemos declarar variáveis a qualquer momento. Porém, dependendo de onde você as declarou, ela vai valer de um determinado ponto a outro.

```
// aqui a variável i não existe
int i = 5;
// a partir daqui ela existe
```

ESCOPO DAS VARIÁVEIS

O escopo da variável é o nome dado ao trecho de código em que aquela variável existe e onde é possível acessá-la. Quando abrimos um novo bloco com as chaves, as variáveis declaradas ali dentro só valem até o fim daquele bloco.

```
// aqui a variável i não existe
int i = 5;
// a partir daqui ela existe
while (condicao) {
    // o i ainda vale aqui
    int j = 7;
    // o j passa a existir
}
// aqui o j não existe mais, mas o i continua dentro do escopo
```

No bloco acima, a variável j pára de existir quando termina o bloco onde ela foi declarada. Se você tentar acessar uma variável fora de seu escopo, ocorrerá um erro de compilação.

```
EscopoDeVariavel.java:8: cannot find symbol
symbol  : variable j
location: class EscopoDeVariavel
                System.out.println(j);
                                   ^
1 error
```

ESCOPO DAS VARIÁVEIS

O mesmo vale para um if:

```
if (algumBooleano) {
    int i = 5;
}
else {
    int i = 10;
}
System.out.println(i); // cuidado!
```

Aqui a variável i não existe fora do if e do else ! Se você declarar a variável antes do if , vai haver outro erro de compilação: dentro do if e do else a variável está sendo redeclarada! Então o código para compilar e fazer sentido fica:

```
int i;
if (algumBooleano) {
    i = 5;
}
else {
    i = 10;
}
System.out.println(i);
```

ESCOPO DAS VARIÁVEIS

Uma situação parecida pode ocorrer com o for :

```
int i;
for (i = 0; i < 10; i++) {
    System.out.println("olá!");
}
System.out.println(i);
```

UM BLOCO DENTRO DO OUTRO

Um bloco também pode ser declarado dentro de outro. Isto é, um if dentro de um for , ou um for dentro de um for , algo como:

```
while (condicao) {
    for (int i = 0; i < 10; i++) {
        // código
    }
}
```

EXERCÍCIOS: FIXAÇÃO DE SINTAXE

Para cada exercício, crie um novo arquivo com extensão .java, e declare aquele estranho cabeçalho, dando nome a uma classe e com um bloco main dentro dele:

1. Imprima todos os números de 150 a 300.
2. Imprima a soma de 1 até 1000.
3. Imprima todos os múltiplos de 3, entre 1 e 100.
4. Imprima os fatoriais de 1 a 10.

   O fatorial de um número n é n * (n-1) * (n-2) * ... * 1. Lembre-se de utilizar os parênteses.
   O fatorial de 0 é 1
   O fatorial de 1 é (0!) * 1 = 1
   O fatorial de 2 é (1!) * 2 = 2
   O fatorial de 3 é (2!) * 3 = 6
   O fatorial de 4 é (3!) * 4 = 24
   Faça um for que inicie uma variável n (número)
   como 1 e fatorial (resultado) como 1 e varia n de 1 até 10:

```
int fatorial = 1;
for (int n = 1; n <= 10; n++) {

}
```

5. No código do exercício anterior, aumente a quantidade de números que terão os fatoriais impressos, até 20, 30, 40. Em um determinado momento, além desse cálculo demorar, vai começar a mostrar respostas completamente erradas. Por quê?

   Mude de int para long para ver alguma mudança.

6. Imprima os primeiros números da série de Fibonacci até passar de 100. A série de Fibonacci é a seguinte: 0, 1, 1, 2, 3, 5, 8, 13, 21, etc... Para calculá-la, o primeiro elemento vale 0, o segundo vale 1, daí por diante, o n-ésimo elemento vale o (n-1)-ésimo elemento somado ao (n-2)-ésimo elemento (ex: 8 = 5 + 3).

7. Escreva um programa que, dada uma variável x com algum valor inteiro, temos um novo x de acordo com a seguinte regra:
   - se x é par, x = x / 2;
   - se x é impar, x = 3 * x + 1;
   - imprime x;
   - O programa deve parar quando x tiver o valor final de 1. Por exemplo, para x = 13 , a saída será: 40 -> 20 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
8. Imprima a seguinte tabela, usando for s encadeados:

```
1
2 4
3 6 9
4 8 12 16
n n*2 n*3 .... n*n
```